EX. NO:14	MINI PROJECT
DATE:23/08/2026	HOSPITAL PATIENT REGISTRATION AND BILLING

Aim

To develop a Diabetes Risk Analysis and Patient Monitoring System using SQL and PL/SQL to store, manage, process, and analyze patient glucose, BMI, blood pressure, and diabetes outcome records.

Software Used

- GitHub Codespaces
- Oracle Database
- SQL
- PL/SQL

Description

The **Diabetes Risk Analysis and Patient Monitoring System** is developed to examine glucose, BMI, blood pressure, and diabetes outcome records of patients using the Pima Indians Diabetes dataset.

The system stores patient records using SQL. SQL queries are used to display patient records, identify patients with a positive diabetes outcome, calculate average glucose levels for each outcome group, retrieve patients with high glucose readings, and compare average BMI. PL/SQL functions and procedures are used to classify glucose levels, retrieve the measurements and outcome of a patient record, and generate a dataset summary while handling invalid patient IDs and missing measurements.

Database Tables

Column	Data Type	Constraint
PATIENT_ID	**NUMBER**	**PRIMARY KEY**
AGE	**NUMBER(3)**	**NOT NULL**
GLUCOSE	**NUMBER(5,1)**	**—**
BLOOD_PRESSURE	**NUMBER(5,1)**	**—**
BMI	**NUMBER(4,1)**	**—**
DIABETES_OUTCOME	**NUMBER(1)**	**NOT NULL (0 or 1)**

1. Table Creation

```
CREATE TABLE diabetes_patients (
  patient_id NUMBER PRIMARY KEY,
  age NUMBER(3) NOT NULL,
  glucose NUMBER(5,1),
  blood_pressure NUMBER(5,1),
  bmi NUMBER(4,1),
  diabetes_outcome NUMBER(1) NOT NULL
    CHECK (diabetes_outcome IN (0,1))
);
```

Table Created

The DIABETES_PATIENTS table was created successfully with primary key, NOT NULL and CHECK constraints.

2.Dataset Insertion — 25 Records

```
INSERT INTO diabetes_patients VALUES (101, 50, 148, 72, 33.6, 1);
INSERT INTO diabetes_patients VALUES (102, 31, 85, 66, 26.6, 0);
INSERT INTO diabetes_patients VALUES (103, 32, 183, 64, 23.3, 1);
INSERT INTO diabetes_patients VALUES (104, 21, 89, 66, 28.1, 0);
INSERT INTO diabetes_patients VALUES (105, 33, 137, 40, 43.1, 1);
INSERT INTO diabetes_patients VALUES (106, 30, 116, 74, 25.6, 0);
INSERT INTO diabetes_patients VALUES (107, 26, 78, 50, 31.0, 1);
INSERT INTO diabetes_patients VALUES (108, 29, 115, 70, 35.3, 0);
INSERT INTO diabetes_patients VALUES (109, 53, 197, 70, 30.5, 1);
INSERT INTO diabetes_patients VALUES (110, 54, 125, 96, 32.3, 1);
INSERT INTO diabetes_patients VALUES (111, 30, 110, 92, 37.6, 0);
INSERT INTO diabetes_patients VALUES (112, 34, 168, 74, 38.0, 1);
INSERT INTO diabetes_patients VALUES (113, 57, 139, 80, 27.1, 0);
INSERT INTO diabetes_patients VALUES (114, 59, 189, 60, 30.1, 1);
INSERT INTO diabetes_patients VALUES (115, 51, 166, 72, 25.8, 1);
INSERT INTO diabetes_patients VALUES (116, 32, 100, 88, 30.0, 0);
INSERT INTO diabetes_patients VALUES (117, 31, 118, 84, 45.8, 1);
INSERT INTO diabetes_patients VALUES (118, 31, 107, 74, 29.6, 1);
```

```
INSERT INTO diabetes_patients VALUES (119, 33, 103, 30, 43.3, 0);
INSERT INTO diabetes_patients VALUES (120, 27, 115, 70, 34.6, 0);
INSERT INTO diabetes_patients VALUES (121, 50, 126, 88, 39.3, 0);
INSERT INTO diabetes_patients VALUES (122, 41, 99, 84, 35.4, 0);
INSERT INTO diabetes_patients VALUES (123, 29, 196, 90, 39.8, 1);
INSERT INTO diabetes_patients VALUES (124, 51, 119, 80, 29.0, 1);
INSERT INTO diabetes_patients VALUES (125, 43, 143, 94, 36.6, 1);
COMMIT;
```

Records Inserted

A total of 25 patient records were inserted into the DIABETES_PATIENTS table.

Query 1 — Display all patients

```
SELECT patient_id, age, glucose,
    blood_pressure, bmi, diabetes_outcome
FROM diabetes_patients
ORDER BY patient_id;
```

Output

```
PATIENT_ID   AGE  GLUCOSE  BLOOD_PRESSURE   BMI  OUTCOME
     101    50     148           72  33.6       1
     102    31      85           66  26.6       0
     103    32     183           64  23.3       1
     104    21      89           66  28.1       0
     105    33     137           40  43.1       1
     106    30     116           74  25.6       0
     107    26      78           50  31.0       1
     108    29     115           70  35.3       0
     109    53     197           70  30.5       1
     110    54     125           96  32.3       1
     111    30     110           92  37.6       0
     112    34     168           74  38.0       1
     113    57     139           80  27.1       0
     114    59     189           60  30.1       1
     115    51     166           72  25.8       1
     116    32     100           88  30.0       0
     117    31     118           84  45.8       1
     118    31     107           74  29.6       1
     119    33     103           30  43.3       0
     120    27     115           70  34.6       0
     121    50     126           88  39.3       0
     122    41      99           84  35.4       0
     123    29     196           90  39.8       1
     124    51     119           80  29.0       1
     125    43     143           94  36.6       1


25 rows selected.
```

Query 2 — Identify records with a positive diabetes outcome

```
SELECT patient_id, age, glucose,
    blood_pressure, bmi, diabetes_outcome
FROM diabetes_patients
WHERE diabetes_outcome = 1
ORDER BY patient_id;
```

Output

```
PATIENT_ID   AGE  GLUCOSE  BLOOD_PRESSURE   BMI  OUTCOME
     101    50     148            72  33.6       1
     103    32     183            64  23.3       1
     105    33     137            40  43.1       1
     107    26      78            50  31.0       1
     109    53     197            70  30.5       1
     110    54     125            96  32.3       1
     112    34     168            74  38.0       1
     114    59     189            60  30.1       1
     115    51     166            72  25.8       1
     117    31     118            84  45.8       1
     118    31     107            74  29.6       1
     123    29     196            90  39.8       1
     124    51     119            80  29.0       1
     125    43     143            94  36.6       1

14 rows selected.
```

Query 3: Calculate average glucose level for each diabetes outcome group

```
SELECT diabetes_outcome,
    COUNT(*) AS total_patients,
    ROUND(AVG(glucose), 2) AS avg_glucose
FROM diabetes_patients
GROUP BY diabetes_outcome
ORDER BY diabetes_outcome;
```

Output

```
DIABETES_OUTCOME  TOTAL_PATIENTS  AVG_GLUCOSE
----------------  --------------  -----------
              0              11       108.82
              1              14       148.14

2 rows selected.
```

Query 4 — Retrieve patients with glucose readings above 140

```
SELECT patient_id, age, glucose,
    blood_pressure, bmi, diabetes_outcome
FROM diabetes_patients
WHERE glucose > 140
ORDER BY glucose DESC;
```

```
PATIENT_ID   AGE  GLUCOSE  BLOOD_PRESSURE   BMI  OUTCOME
     109    53     197            70  30.5       1
     123    29     196            90  39.8       1
     114    59     189            60  30.1       1
     103    32     183            64  23.3       1
     112    34     168            74  38.0       1
     115    51     166            72  25.8       1
     101    50     148            72  33.6       1
     125    43     143            94  36.6       1

8 rows selected.
```

Query 5: Calculate average BMI for patients with positive and negative diabetes outcomes

```
SELECT diabetes_outcome,
    ROUND(AVG(bmi), 2) AS avg_bmi
FROM diabetes_patients
GROUP BY diabetes_outcome
ORDER BY diabetes_outcome;
```

Output

```
DIABETES_OUTCOME  AVG_BMI
----------------  -------
              0    32.99
              1    33.46

2 rows selected.
```

PL/SQL Procedure 1:

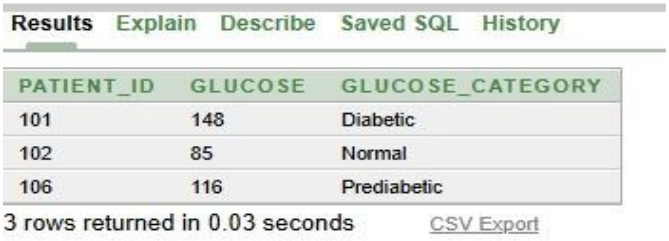
```
CREATE OR REPLACE FUNCTION classify_glucose (
  p_glucose IN NUMBER
)
RETURN VARCHAR2
IS
```

```
BEGIN
  IF p_glucose IS NULL THEN
    RETURN 'Not Recorded';
  ELSIF p_glucose < 100 THEN
    RETURN 'Normal';
  ELSIF p_glucose <= 125 THEN
    RETURN 'Prediabetic';
  ELSE
    RETURN 'Diabetic';
  END IF;
END;
/
```

Execution

```
SELECT patient_id,
       glucose,
       classify_glucose(glucose) AS glucose_category
FROM diabetes_patients
WHERE patient_id IN (101, 102, 106);
```

Output :



Results Explain Describe Saved SQL History

PATIENT_ID	GLUCOSE	GLUCOSE_CATEGORY
101	148	Diabetic
102	85	Normal
106	116	Prediabetic

3 rows returned in 0.03 seconds CSV Export

PL/SQL Function 2: Retrieve Patient Measurements and Outcome

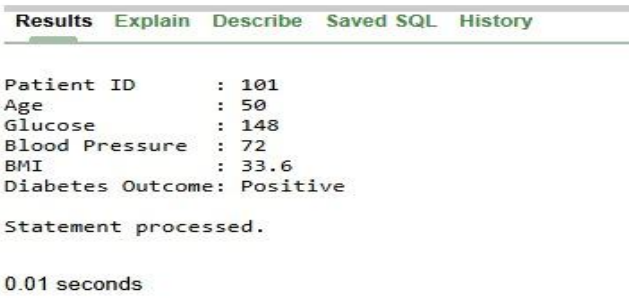
```
CREATE OR REPLACE PROCEDURE get_patient_record (
  p_patient_id IN NUMBER
)
IS
  v_age     diabetes_patients.age%TYPE;
  v_glucose diabetes_patients.glucose%TYPE;
  v_bp      diabetes_patients.blood_pressure%TYPE;
```

```
  v_bmi     diabetes_patients.bmi%TYPE;
  v_outcome diabetes_patients.diabetes_outcome%TYPE;
BEGIN
  SELECT age, glucose, blood_pressure, bmi, diabetes_outcome
  INTO v_age, v_glucose, v_bp, v_bmi, v_outcome
  FROM diabetes_patients
  WHERE patient_id = p_patient_id;
  DBMS_OUTPUT.PUT_LINE('Patient ID      : ' || p_patient_id);
  DBMS_OUTPUT.PUT_LINE('Age             : ' || v_age);
  DBMS_OUTPUT.PUT_LINE('Glucose         : ' || NVL(TO_CHAR(v_glucose), 'Not recorded'));
  DBMS_OUTPUT.PUT_LINE('Blood Pressure  : ' || NVL(TO_CHAR(v_bp), 'Not recorded'));
  DBMS_OUTPUT.PUT_LINE('BMI             : ' || NVL(TO_CHAR(v_bmi), 'Not recorded'));
  DBMS_OUTPUT.PUT_LINE('Diabetes Outcome: ' ||
    CASE v_outcome WHEN 1 THEN 'Positive' ELSE 'Negative' END);
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Patient ID not found.');
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

Execution

```
SET SERVEROUTPUT ON;
BEGIN
  get_patient_record(101);
END;
/
```

Output



PL/SQL Procedure 3: Generate Diabetes Dataset Summary

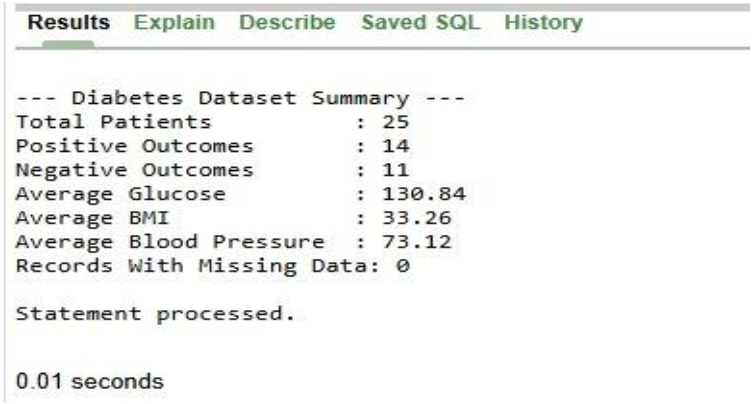
```
CREATE OR REPLACE PROCEDURE generate_diabetes_summary (
  p_patient_id IN NUMBER DEFAULT NULL
)
IS
  v_total    NUMBER;
  v_positive NUMBER;
  v_negative NUMBER;
  v_avg_glu  NUMBER;
  v_avg_bmi  NUMBER;
  v_avg_bp   NUMBER;
  v_missing  NUMBER;
  v_exists   NUMBER;
BEGIN
  -- Validate patient ID when one is supplied
  IF p_patient_id IS NOT NULL THEN
    SELECT COUNT(*) INTO v_exists
    FROM diabetes_patients
    WHERE patient_id = p_patient_id;
    IF v_exists = 0 THEN
      DBMS_OUTPUT.PUT_LINE('Patient ID not found.');
      RETURN;
    END IF;
  END IF;
  SELECT COUNT(*),
        NVL(SUM(CASE WHEN diabetes_outcome = 1 THEN 1 ELSE 0 END), 0),
        NVL(SUM(CASE WHEN diabetes_outcome = 0 THEN 1 ELSE 0 END), 0),
        ROUND(AVG(glucose), 2),
        ROUND(AVG(bmi), 2),
        ROUND(AVG(blood_pressure), 2),
        NVL(SUM(CASE WHEN glucose IS NULL
                  OR bmi IS NULL
                  OR blood_pressure IS NULL
                 THEN 1 ELSE 0 END), 0)
  INTO v_total, v_positive, v_negative,
```

```
    v_avg_glu, v_avg_bmi, v_avg_bp, v_missing
  FROM diabetes_patients;
  IF v_total = 0 THEN
    DBMS_OUTPUT.PUT_LINE('No patient records available.');
    RETURN;
  END IF;
  DBMS_OUTPUT.PUT_LINE('--- Diabetes Dataset Summary ---');
  DBMS_OUTPUT.PUT_LINE('Total Patients          : ' || v_total);
  DBMS_OUTPUT.PUT_LINE('Positive Outcomes       : ' || v_positive);
  DBMS_OUTPUT.PUT_LINE('Negative Outcomes       : ' || v_negative);
  DBMS_OUTPUT.PUT_LINE('Average Glucose         : ' || NVL(TO_CHAR(v_avg_glu), 'N/A'));
  DBMS_OUTPUT.PUT_LINE('Average BMI             : ' || NVL(TO_CHAR(v_avg_bmi), 'N/A'));
  DBMS_OUTPUT.PUT_LINE('Average Blood Pressure  : ' || NVL(TO_CHAR(v_avg_bp), 'N/A'));
  DBMS_OUTPUT.PUT_LINE('Records With Missing Data: ' || v_missing);
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

Execution

```
BEGIN
  generate_diabetes_summary;
END;
/
```

Output



1. Glucose Category Report

Query

```
SELECT classify_glucose(glucose) AS glucose_category,
    COUNT(*) AS total_patients,
    ROUND(AVG(bmi), 2) AS avg_bmi
FROM diabetes_patients
GROUP BY classify_glucose(glucose)
ORDER BY total_patients DESC;
```

Output



Results Explain Describe Saved SQL History

GLUCOSE_CATEGORY	TOTAL_PATIENTS	AVG_BMI
Diabetic	11	33.38
Prediabetic	10	34.31
Normal	4	30.28

3 rows returned in 0.00 seconds CSV Export

2. Exception Handling

Test Case 1 – Invalid Patient ID for Patient Record Retrieval

```
BEGIN
  get_patient_record(999);
END;
/
```

Output



Results Explain Describe Saved SQL History

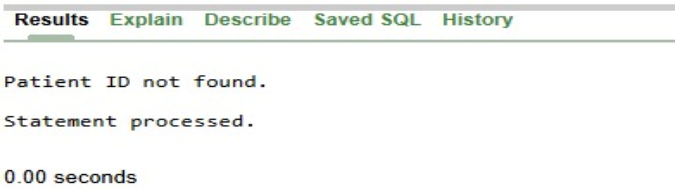
```
Patient ID not found.
Statement processed.

0.00 seconds
```

Test Case 2 – Invalid Patient ID for Dataset Summary

```
BEGIN
  generate_diabetes_summary(999);
END;
/
```

Output



Test Case 3 – Missing Glucose Measurement

```
SELECT classify_glucose(NULL) AS glucose_category
FROM dual;
```

Output



Dataset Insights

- The dataset contains 25 patient records from the Pima Indians Diabetes dataset.
- 14 patients have a positive diabetes outcome and 11 have a negative outcome.
- The average glucose of positive patients (148.14) is clearly higher than that of negative patients (108.82).
- The average BMI is similar for both groups (33.46 positive, 32.99 negative).
- 8 patients have a glucose reading above 140, and all of them have a positive outcome.
- Glucose categories help classify patients as Normal, Prediabetic, or Diabetic for monitoring.

4. Result

Thus, the Diabetes Risk Analysis and Patient Monitoring System was successfully implemented using SQL and PL/SQL. The diabetes patient table was created, 25 patient records were inserted, SQL queries were executed, PL/SQL functions and procedures were implemented, and exception handling for invalid patient IDs and missing measurements was tested successfully.